

Day 07 - Git Revert

Objective

Understand Git Revert, rollback workflows, merge revert, and production best practices.

What is Git Revert?

Git Revert creates a new commit that reverses the changes introduced by an earlier commit while preserving project history.

Reset vs Revert

Git Reset rewrites history by moving HEAD. Git Revert preserves history by creating a new rollback commit. Use Revert for shared branches.

Internal Working

Target Commit → Generate Reverse Patch → Apply Reverse Changes → Create New Commit → Move HEAD Forward.

Important Commands

```
git revert HEAD
git revert <commit>
git revert --no-commit <commit>
git revert -m 1 <merge_commit_hash>
git revert --continue
git revert --abort
```

Production Workflow

Bug Found → Identify Commit → git show → git revert → Test → Push → Production Restored.

Merge Commit

Use git revert -m 1 when reverting a merge commit. Parent 1 is treated as the mainline branch.

Hands-on Labs

1. Revert latest commit
2. Revert older commit
3. Revert using --no-commit
4. Revert merge commit

Best Practices

Inspect commits before reverting, test locally, prefer Revert over Reset for shared branches, and avoid force-pushing published history.

Interview Questions

What is Git Revert?
Reset vs Revert?
What does -m 1 mean?
Why create a new commit?

When would you use --no-commit?